Niveau : T-IINFO-GL **Année universitaire :** 2026/2027	**Matière :** Framework Web Front-End **Enseignante :** Mme Eya MAAZOUN	**ISIMS**

COMPTE RENDU

TP 2 : Components et Data Binding

Application TaskFlow – Angular 22, Standalone Components, Signals, RxJS et Tailwind CSS

Réalisé par : **Hassen KRICHEN**

Enseignante : **Mme Eya MAAZOUN**

Classe : **T-IINFO-GL**

Année universitaire : **2026/2027**

Date : 29 septembre 2026

Table des matières

Introduction 4
Environnement de travail 4
Partie 1 – Création du projet 5
Étape 1 : Présentation de l'application 5
Étape 2 : Génération du projet avec Angular CLI 5
Étape 3 : Installation de Tailwind CSS 5
Partie 2 – Architecture du projet 7
Étape 1 : Choix de l'architecture 7
Étape 2 : Arborescence obtenue 7
Partie 3 – Création des modèles 8
Étape 1 : Types personnalisés et interfaces 8
Partie 4 – Création des données 9
Partie 5 – Création des composants 10
Étape 1 : Composant ProjectList 10
Étape 2 : Composant ProjectCard 11
Étape 3 : Composant TaskList 12
Étape 4 : Composant partagé EmptyState 13
Étape 5 : Intégration dans le composant racine 14
Partie 6 – Data Binding 15
Étape 1 : Interpolation 15
Étape 2 : Property Binding (ProjectList → ProjectCard) 15
Étape 3 : Input entre composants (ProjectCard → TaskList) 16
Étape 4 : Event Binding 16
Étape 5 : Output (ProjectCard → ProjectList) 17
Synthèse des types de binding utilisés 18
Partie 7 – Two-Way Data Binding 19
Étape 1 : Champ de recherche avec [(ngModel)] 19
Étape 2 : Résultat 19
Partie 8 – Nouvelle syntaxe de contrôle Angular 21
Étape 1 : Boucle @for 21
Étape 2 : Conditions @if / @else 21
Partie 9 – Style avec Tailwind CSS 23
Étape 1 : Application des consignes 23
Étape 2 : Rendu responsive 24
Partie 10 – Introduction aux Signals 25
Étape 1 : Remplacer le tableau par un Signal 25
Étape 2 : Lecture et modification 25

Partie 11 – Computed Signals 27
Étape 1 : Signal de recherche 27
Étape 2 : Signal calculé filteredProjects 27
Étape 3 : Utilisation dans le template 27
Partie 12 – Introduction à RxJS 28
Étape 1 et 2 : Service et méthode getProjects() 28
Étape 3 : Données statiques avec of() 28
Étape 4 : Subscription explicite dans ProjectList 28
Tests et validation 29
Compilation 29
Tests unitaires 29
Vérification fonctionnelle 30
Difficultés rencontrées 30
Conclusion 31

Introduction

Ce compte rendu présente la réalisation du TP 2 du module **Framework Web Front-End**, consacré aux composants et au Data Binding avec Angular. L'objectif est de construire, étape par étape, l'application **TaskFlow** : une application qui affiche une liste de projets, chacun décrit par un nom, une description, un statut et une liste de tâches (titre, priorité et statut).

Le travail couvre les points suivants :

- la création d'un projet Angular 22 basé sur des composants **Standalone** ;
- l'organisation du code selon une architecture **feature-based** ;
- la communication parent/enfant avec les API modernes `input()` et `output()` ;
- les différents types de **Data Binding** (interpolation, property binding, event binding, two-way binding) ;
- la nouvelle syntaxe de contrôle `@if` / `@for` ;
- la mise en forme avec **Tailwind CSS** ;
- les **Signals** (`signal()`, `computed()`) et une première utilisation de **RxJS** avec `of()`.

Pour chaque partie de l'énoncé, nous présentons la démarche suivie, un extrait du code source (les numéros de ligne affichés sont ceux des fichiers réels) et, lorsque c'est pertinent, une capture de l'interface obtenue.

Environnement de travail

Outil	Version	Rôle
Angular (core, CLI, build)	22.2	Framework front-end et outillage
TypeScript	6.0	Langage typé du projet
Tailwind CSS	4.3	Framework CSS utilitaire
RxJS	7.8	Programmation réactive (Observables)
Vitest	5.0	Exécution des tests unitaires
Node.js / npm	24.21 / 11	Environnement d'exécution et gestionnaire de paquets

Partie 1 – Création du projet

Étape 1 : Présentation de l'application

TaskFlow affiche une liste de projets. Chaque projet possède un nom, une description, un statut et une liste de tâches ; chaque tâche possède un titre, une priorité et un statut. L'utilisateur peut rechercher un projet par son nom, trier la liste et sélectionner un projet.

Étape 2 : Génération du projet avec Angular CLI

Le projet a été généré avec Angular CLI :

```
ng new TaskFlow
```

Depuis Angular 17, les composants générés sont **Standalone** par défaut : il n'y a plus de `NgModule`, chaque composant déclare lui-même ses dépendances dans son tableau `imports`. Le fichier `package.json` confirme l'utilisation d'Angular 22 et de TypeScript :

package.json · lignes 13–34

```
13   "dependencies": {
14     "@angular/common": "^22.2.0",
15     "@angular/compiler": "^22.2.0",
16     "@angular/core": "^22.2.0",
17     "@angular/forms": "^22.2.0",
18     "@angular/platform-browser": "^22.2.0",
19     "@angular/router": "^22.2.0",
20     "rxjs": "~7.8.0",
21     "tslib": "^2.3.0"
22   },
23   "devDependencies": {
24     "@angular/build": "^22.2.0",
25     "@angular/cli": "^22.2.0",
26     "@angular/compiler-cli": "^22.2.0",
27     "@tailwindcss/postcss": "^4.3.3",
28     "jsdom": "^30.0.0",
29     "postcss": "^8.5.28",
30     "prettier": "^3.8.1",
31     "tailwindcss": "^4.3.3",
32     "typescript": "~6.0.2",
33     "vitest": "^5.0.0"
34   }
```

***Figure 1** – Dépendances du projet dans `package.json` (Angular 22, RxJS, Tailwind CSS, TypeScript)*

Étape 3 : Installation de Tailwind CSS

Tailwind CSS (version 4) est installé avec son plugin PostCSS :

Windows PowerShell

```
PS C:\...\Angular-TP\TaskFlow> npm install tailwindcss @tailwindcss/postcss
postcss --save-dev

added 14 packages, and audited 282 packages in 16s

found 0 vulnerabilities
```

***Figure 2** – Installation de Tailwind CSS et de PostCSS*

La configuration se limite à deux fichiers : `.postcssrc.json`, qui active le plugin `@tailwindcss/postcss` (détecté automatiquement par le builder Angular), et `src/styles.css`, qui importe Tailwind. Avec Tailwind 4, aucun fichier `tailwind.config.js` n'est nécessaire : les classes utilisées dans les templates sont détectées automatiquement.

.postcssrc.json

```
{
  "plugins": {
    "@tailwindcss/postcss": {}
  }
}
```

styles.css — src

```
@import 'tailwindcss';
```

***Figure 3** – Configuration de Tailwind CSS*

Partie 2 – Architecture du projet

Étape 1 : Choix de l'architecture

L'énoncé propose deux architectures. TaskFlow ne comporte pour l'instant qu'une seule fonctionnalité (les projets), peu de composants et aucune route : nous avons donc retenu l'**architecture simple** (Example 1). Elle reste néanmoins feature-based, ce qui permettra d'évoluer vers l'architecture scalable (pages, routes et services par fonctionnalité) lorsque d'autres fonctionnalités seront ajoutées (tickets, tableaux, utilisateurs…).

Étape 2 : Arborescence obtenue



***Figure 4** – Arborescence du dossier `src/` de TaskFlow*

Dossier	Rôle
`core/services/`	Services uniques pour toute l'application (`ProjectService`).
`shared/components/`	Composants réutilisables sans logique métier (`EmptyState`).
`features/projects/models/`	Interfaces TypeScript `Task` et `Project`.
`features/projects/data/`	Données statiques `PROJECTS`.
`features/projects/components/`	Composants de la fonctionnalité : `ProjectList`, `ProjectCard`, `TaskList`.
`features/projects/state/`	Réservé au futur store (`project.store.ts`) ; non utilisé dans ce TP.

Partie 3 – Création des modèles

Étape 1 : Types personnalisés et interfaces

Les valeurs autorisées pour la priorité et le statut sont définies par des **types union littéraux** : `TaskPriority` (Haute | Moyenne | Basse) et `Status` (En attente | En cours | Terminé). Le type `Status` est partagé entre `Task` et `Project`, puisque les deux utilisent les mêmes valeurs. Toute valeur non prévue (par exemple « Urgent ») est rejetée dès la compilation.

task.model.ts — src/app/features/projects/models

```ts
export type TaskPriority = 'Haute' | 'Moyenne' | 'Basse';

export type Status = 'En attente' | 'En cours' | 'Terminé';

export interface Task {
  id: number;
  title: string;
  priority: TaskPriority;
  status: Status;
}
```

project.model.ts — src/app/features/projects/models

```ts
import { Status, Task } from './task.model';

export interface Project {
  id: number;
  name: string;
  description: string;
  status: Status;
  tasks: Task[];
}
```

***Figure 5** – Fichiers `task.model.ts` et `project.model.ts`*

Partie 4 – Création des données

Le fichier `projects.data.ts` exporte la constante `PROJECTS`, typée `Project[]` : TypeScript vérifie ainsi que chaque projet et chaque tâche respectent les modèles de la partie 3. Il contient **quatre projets** (le minimum demandé est trois). Le quatrième, « Tableau de bord RH », n'a volontairement **aucune tâche** afin de tester l'affichage « Aucune tâche » (partie 8).

projects.data.ts — src/app/features/projects/data · lignes 1–14

```ts
import { Project } from '../models/project.model';

export const PROJECTS: Project[] = [
  {
    id: 1,
    name: 'Site e-commerce',
    description: 'Refonte de la boutique en ligne avec un nouveau tunnel de paiement.',
    status: 'En cours',
    tasks: [
      { id: 1, title: 'Maquettes des pages produit', priority: 'Haute', status: 'Terminé' },
      { id: 2, title: 'Intégration du paiement', priority: 'Haute', status: 'En cours' },
      { id: 3, title: 'Tests de charge', priority: 'Moyenne', status: 'En attente' },
    ],
  },
```

***Figure 6** – Début du fichier `projects.data.ts` (premier projet et ses trois tâches)*

Partie 5 – Création des composants

L'interface est découpée en composants Standalone, chacun ayant une responsabilité unique :

Composant	Sélecteur	Responsabilité	Entrées / sorties
`ProjectList`	`app-project-list`	Récupère les projets, gère la recherche, le tri et la sélection.	—
`ProjectCard`	`app-project-card`	Affiche un projet, son statut et ses tâches ; permet la sélection.	in : `project`, `selected` · out : `projectSelected`
`TaskList`	`app-task-list`	Affiche les tâches, leur priorité et leur statut avec un style par statut.	in : `tasks`
`EmptyState`	`app-empty-state`	Message réutilisable quand une liste est vide.	in : `message`

Étape 1 : Composant ProjectList

`ProjectList` est le composant parent. Il importe `FormsModule` (pour `ngModel`), `ProjectCard` et `EmptyState`, et contient l'état de la page sous forme de signals : la liste des projets, le texte recherché et le projet sélectionné.

project-list.ts — src/app/features/projects/components/project-list · lignes 9–19

```
 9  @Component({
10    selector: 'app-project-list',
11    imports: [FormsModule, ProjectCard, EmptyState],
12    templateUrl: './project-list.html',
13  })
14  export class ProjectList implements OnInit {
15    private readonly projectService = inject(ProjectService);
16
17    projects = signal<Project[]>(PROJECTS);
18    searchTerm = signal('');
19    selectedProject = signal<Project | null>(null);
```

***Figure 7** – Déclaration du composant `ProjectList` et de son état*

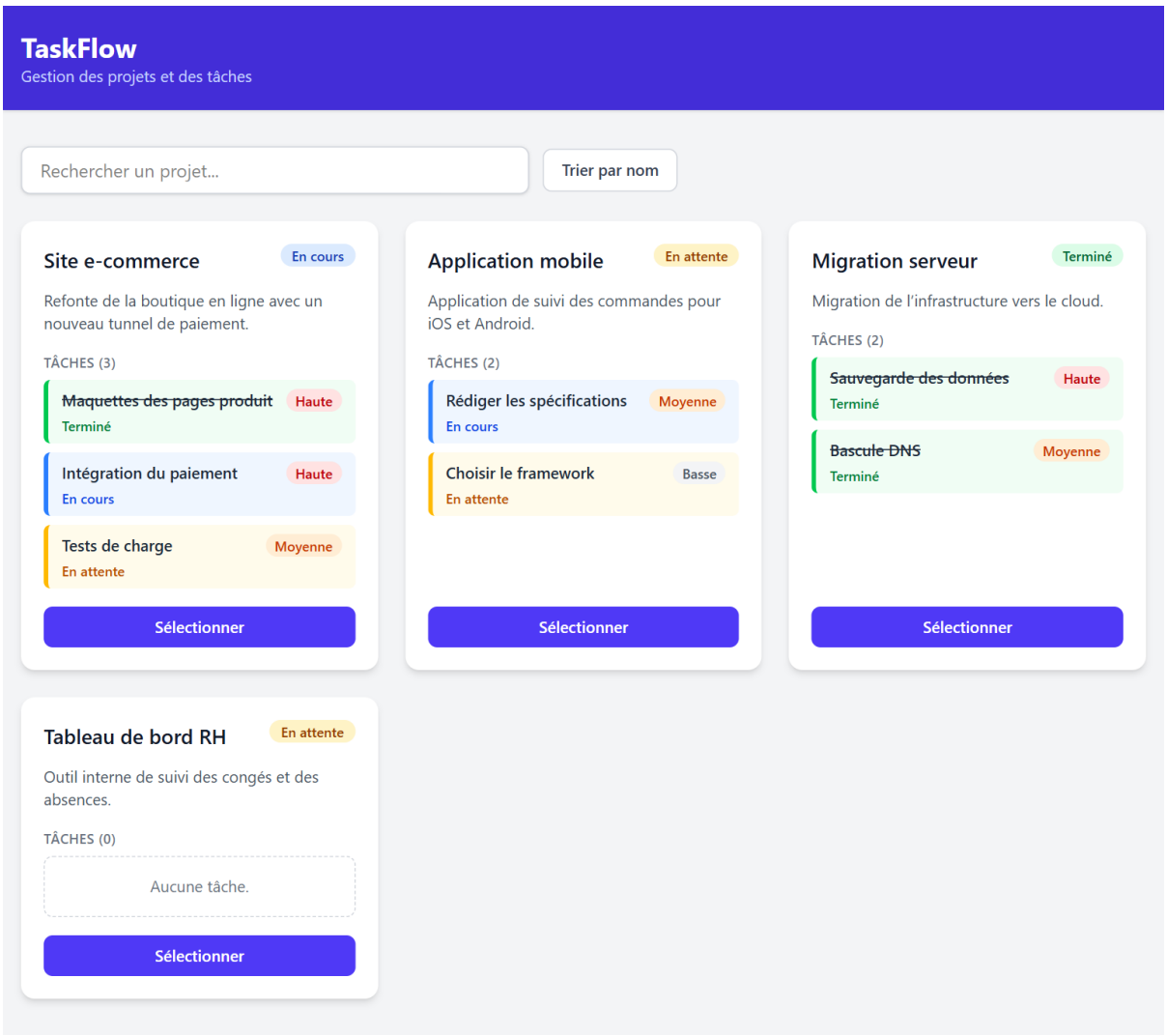


***Figure 8** – Interface générée par `ProjectList` : barre de recherche, bouton de tri et grille de cartes*

Étape 2 : Composant ProjectCard

`ProjectCard` reçoit un projet de son parent et affiche ses informations, son statut (sous forme de badge coloré) et ses tâches. Le bouton « Sélectionner » prévient le parent via une sortie.

project-card.ts — src/app/features/projects/components/project-card · lignes 6–25

```ts
@Component({
  selector: 'app-project-card',
  imports: [TaskList],
  templateUrl: './project-card.html',
})
export class ProjectCard {
  project = input.required<Project>();
  selected = input(false);
  projectSelected = output<Project>();

  protected readonly statusBadge: Record<Status, string> = {
    'En attente': 'bg-amber-100 text-amber-800',
    'En cours': 'bg-blue-100 text-blue-800',
    Terminé: 'bg-green-100 text-green-800',
  };

  selectProject(): void {
    this.projectSelected.emit(this.project());
  }
}
```

***Figure 9** – Classe du composant `ProjectCard`*

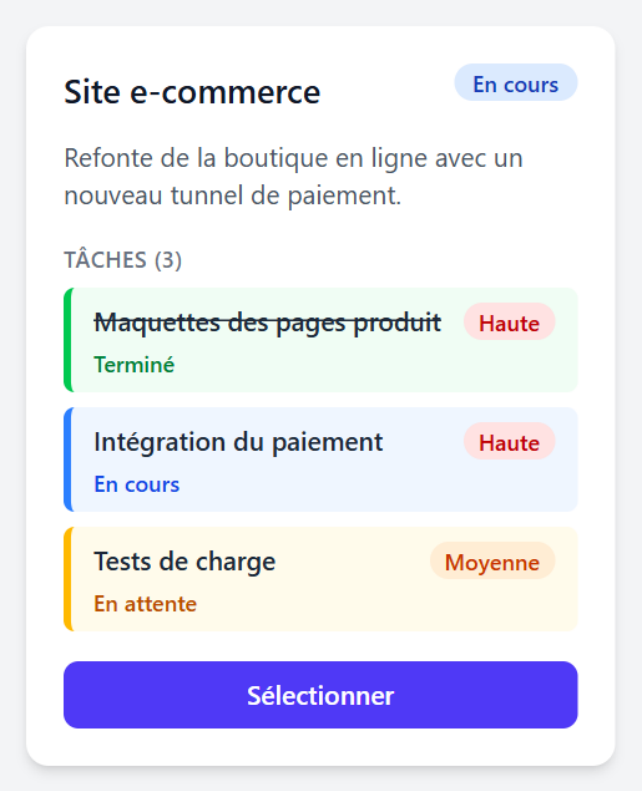


***Figure 10** – Rendu d'une carte de projet*

Étape 3 : Composant TaskList

`TaskList` reçoit le tableau de tâches et associe à chaque statut et à chaque priorité un ensemble de classes Tailwind (détaillé en partie 9). Il importe `EmptyState` pour le cas d'une liste vide.

task-list.ts — src/app/features/projects/components/task-list · lignes 1–11

```ts
import { Component, input } from '@angular/core';
import { EmptyState } from '../../../../shared/components/empty-state/empty-state';
import { Status, Task, TaskPriority } from '../../models/task.model';

@Component({
  selector: 'app-task-list',
  imports: [EmptyState],
  templateUrl: './task-list.html',
})
export class TaskList {
  tasks = input.required<Task[]>();
```

***Figure 11** – Déclaration du composant `TaskList` et de son entrée `tasks`*



***Figure 12** – Liste des tâches : bordure et couleur selon le statut, badge selon la priorité*

Étape 4 : Composant partagé EmptyState

Le message affiché lorsqu'une liste est vide est utilisé à deux endroits (aucune tâche, aucun projet trouvé). Il a donc été placé dans `shared/components/`, avec un template inline et une seule entrée `message`.

empty-state.ts — src/app/shared/components/empty-state

```ts
import { Component, input } from '@angular/core';

@Component({
  selector: 'app-empty-state',
  template: `<p class="rounded-lg border border-dashed border-gray-300 p-4 text-center text-sm text-gray-500">
    {{ message() }}
  </p>`,
})
export class EmptyState {
  message = input.required<string>();
}
```

***Figure 13** – Composant réutilisable `EmptyState`*

Étape 5 : Intégration dans le composant racine

Le composant racine `App` importe `ProjectList` et l'affiche sous un en-tête contenant le titre de l'application.

app.ts — src/app

```ts
import { Component, signal } from '@angular/core';
import { ProjectList } from './features/projects/components/project-list/project-list';

@Component({
  imports: [ProjectList],
  selector: 'app-root',
  styleUrl: './app.css',
  templateUrl: './app.html',
})
export class App {
  protected readonly title = signal('TaskFlow');
}
```

app.html — src/app

```html
<div class="min-h-screen bg-gray-100">
  <header class="bg-indigo-700 shadow">
    <div class="mx-auto max-w-6xl px-4 py-5">
      <h1 class="text-2xl font-bold text-white">{{ title() }}</h1>
      <p class="text-sm text-indigo-200">Gestion des projets et des tâches</p>
    </div>
  </header>

  <main class="mx-auto max-w-6xl px-4 py-8">
    <app-project-list />
  </main>
</div>
```

***Figure 14** – Composant racine : `app.ts` et `app.html`*

Partie 6 – Data Binding

Étape 1 : Interpolation

L'interpolation `{{ … }}` affiche la valeur d'une expression dans la vue. Comme `project` est un input basé sur les signals, il se lit en l'appelant : `project().name`. Elle est utilisée pour le nom, le statut et la description du projet, le nombre de tâches, ainsi que pour le titre, la priorité et le statut de chaque tâche.

project-card.html — src/app/features/projects/components/project-card · lignes 6–17

```
6   <header class="mb-3 flex items-start justify-between gap-3">
7     <h2 class="text-lg font-semibold text-gray-900">{{ project().name }}
  </h2>
8     <span class="shrink-0 rounded-full px-2.5 py-0.5 text-xs font-
  semibold" [class]="statusBadge[project().status]">
9       {{ project().status }}
10    </span>
11  </header>
12
13  <p class="mb-4 text-sm text-gray-600">{{ project().description }}</p>
14
15  <h3 class="mb-2 text-xs font-semibold uppercase tracking-wide text-
  gray-500">
16    Tâches ({{ project().tasks.length }})
17  </h3>
```

task-list.html — src/app/features/projects/components/task-list · lignes 12–18

```
12            {{ task.title }}
13          </span>
14          <span class="rounded-full px-2 py-0.5 text-xs font-semibold"
  [class]="priorityBadge[task.priority]">
15            {{ task.priority }}
16          </span>
17        </div>
18        <p class="mt-1 text-xs font-medium"
  [class]="statusText[task.status]">{{ task.status }}</p>
```

***Figure 15** – Interpolation dans `project-card.html` et `task-list.html`*

Étape 2 : Property Binding (ProjectList → ProjectCard)

Dans `ProjectList`, `[project]="project"` transmet à la carte l'objet de l'itération courante. Côté enfant, `input.required<Project>()` déclare une entrée **obligatoire** : si le parent oublie de la fournir, la compilation échoue. Une seconde entrée, `selected` (valeur par défaut `false`), indique à la carte si elle est sélectionnée.

project-list.html — src/app/features/projects/components/project-list · lignes 28–34

```
28        @for (project of filteredProjects(); track project.id) {
29          <app-project-card
30            [project]="project"
31            [selected]="selectedProject()?.id === project.id"
32            (projectSelected)="onProjectSelected($event)"
33          />
34        }
```

project-card.ts — src/app/features/projects/components/project-card · lignes 11–13

```
11  export class ProjectCard {
12    project = input.required<Project>();
13    selected = input(false);
```

***Figure 16** – Property binding côté parent et déclaration des inputs côté enfant*

Étape 3 : Input entre composants (ProjectCard → TaskList)

Sur le même principe, `ProjectCard` transmet les tâches du projet à `TaskList` via `[tasks]="project().tasks"`.

project-card.html — src/app/features/projects/components/project-card · ligne 18

```
18    <app-task-list [tasks]="project().tasks" />
```

task-list.ts — src/app/features/projects/components/task-list · lignes 10–11

```
10  export class TaskList {
11    tasks = input.required<Task[]>();
```

***Figure 17** – Transmission des tâches à `TaskList`*

Étape 4 : Event Binding

Le bouton « Sélectionner » lie l'événement DOM `click` à la méthode `selectProject()`. On y combine aussi un property binding, `[disabled]="selected()"`, pour désactiver le bouton de la carte déjà sélectionnée, et une interpolation conditionnelle pour son libellé.

project-card.html — src/app/features/projects/components/project-card · lignes 20–29

```
20    <div class="mt-auto pt-4">
21      <button
22        type="button"
23        class="w-full rounded-lg bg-indigo-600 px-4 py-2 text-sm font-
  medium text-white hover:bg-indigo-700 disabled:bg-indigo-300"
24        [disabled]="selected()"
25        (click)="selectProject()"
26      >
27        {{ selected() ? 'Sélectionné' : 'Sélectionner' }}
28      </button>
29    </div>
```

***Figure 18** – Bouton « Sélectionner » : event binding `(click)`*

Étape 5 : Output (ProjectCard → ProjectList)

`ProjectCard` déclare une sortie `projectSelected = output<Project>()`. La méthode `selectProject()` émet le projet courant ; le parent écoute cet événement avec `(projectSelected)="onProjectSelected($event)"`, où `$event` contient le projet émis. Dans `onProjectSelected()`, le projet est affiché dans la console, comme demandé, puis mémorisé dans le signal `selectedProject`, ce qui affiche un bandeau et met en évidence la carte choisie.

project-card.ts — src/app/features/projects/components/project-card · ligne 14

```ts
14    projectSelected = output<Project>();
```

project-card.ts — src/app/features/projects/components/project-card · lignes 22–24

```ts
22    selectProject(): void {
23      this.projectSelected.emit(this.project());
24    }
```

project-list.ts — src/app/features/projects/components/project-list · lignes 32–35

```ts
32    onProjectSelected(project: Project): void {
33      console.log(project);
34      this.selectedProject.set(project);
35    }
```

project-list.html — src/app/features/projects/components/project-list · lignes 18–22

```html
18    @if (selectedProject(); as selected) {
19      <p class="rounded-lg bg-indigo-50 px-4 py-2 text-sm text-indigo-
    800">
20        Projet sélectionné : <strong>{{ selected.name }}</strong>
21      </p>
22    }
```

***Figure 19** – Émission de l'événement par l'enfant et traitement par le parent*

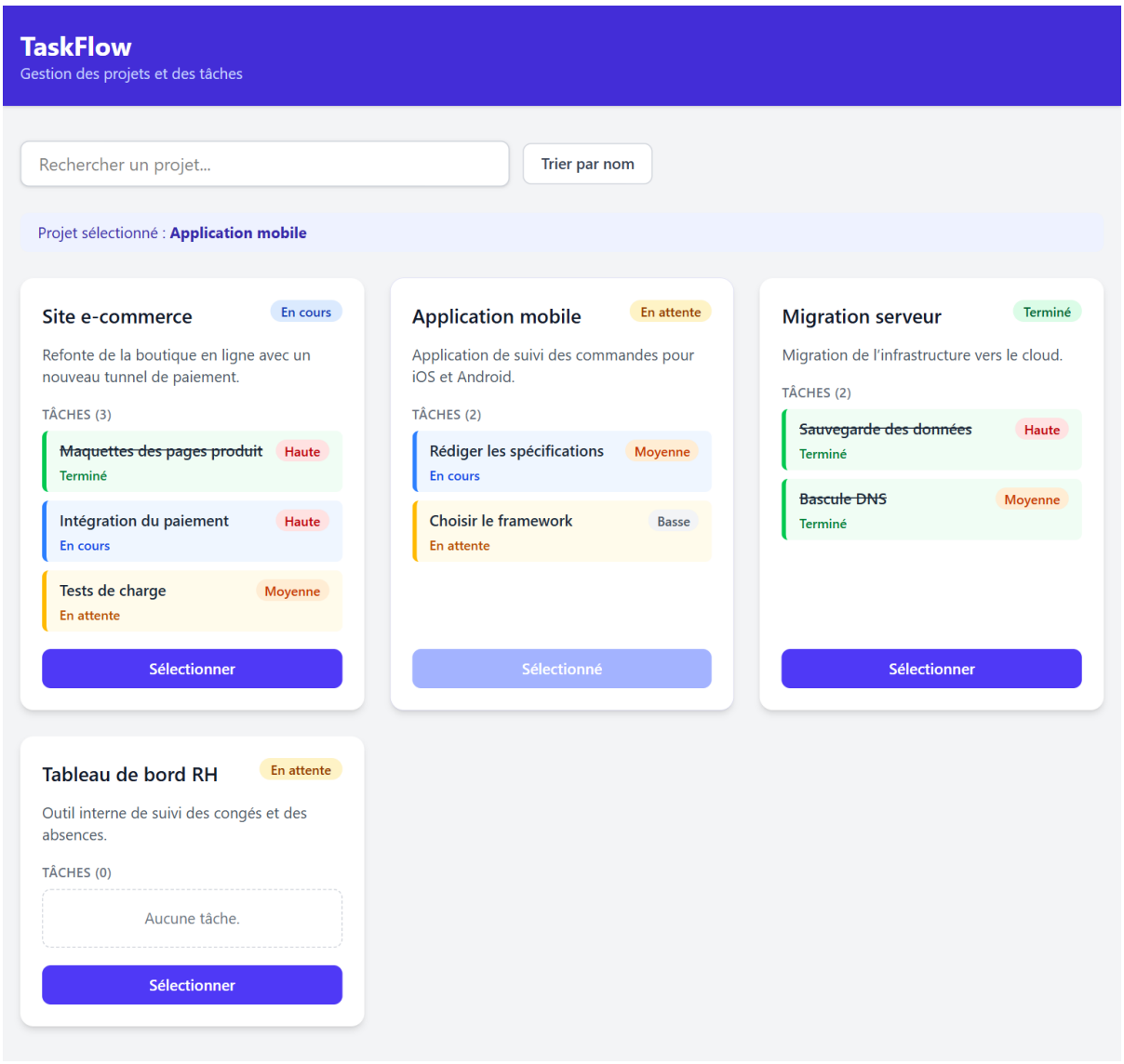


***Figure 20** – Après un clic sur « Sélectionner » : bandeau, carte encadrée et bouton désactivé*

Dans la console du navigateur, `onProjectSelected()` affiche l'objet complet du projet sélectionné (id 2, « Application mobile », avec ses deux tâches). Cela confirme que l'événement transporte bien la donnée de l'enfant vers le parent.

Synthèse des types de binding utilisés

Type	Syntaxe	Sens	Exemple dans TaskFlow
Interpolation	`{{ expr }}`	composant → vue	`{{ project().name }}`
Property binding	`[prop]="expr"`	composant → vue / parent → enfant	`[project]="project"`
Event binding	`(event)="f()"`	vue → composant	`(click)="selectProject()"`
Output	`(out)="f($event)"`	enfant → parent	`(projectSelected)="…"`
Two-way binding	`[(ngModel)]="x"`	dans les deux sens	`[(ngModel)]="searchTerm"`

Partie 7 – Two-Way Data Binding

Étape 1 : Champ de recherche avec [(ngModel)]

La syntaxe « banana in a box » `[(ngModel)]` combine un property binding `[ngModel]` (le modèle vers le champ) et un event binding `(ngModelChange)` (le champ vers le modèle). Elle nécessite l'import de `FormsModule` dans le composant.

Le champ est lié directement au signal `searchTerm` : Angular lit sa valeur pour remplir le champ et appelle `searchTerm.set()` à chaque frappe. La liste filtrée (partie 11) est alors recalculée automatiquement.

project-list.html — src/app/features/projects/components/project-list · lignes 3–8

```
3      <input
4        type="text"
5        placeholder="Rechercher un projet..."
6        class="w-full rounded-lg border border-gray-300 bg-white px-4 py-2
shadow-sm focus:border-indigo-500 focus:outline-none focus:ring-2
focus:ring-indigo-200 sm:max-w-md"
7        [(ngModel)]="searchTerm"
8      />
```

project-list.ts — src/app/features/projects/components/project-list · lignes 1–2

```
1 import { Component, computed, inject, OnInit, signal } from
'@angular/core';
2 import { FormsModule } from '@angular/forms';
```

project-list.ts — src/app/features/projects/components/project-list · lignes 9–13

```
9  @Component({
10   selector: 'app-project-list',
11   imports: [FormsModule, ProjectCard, EmptyState],
12   templateUrl: './project-list.html',
13 })
```

***Figure 21** – Champ de recherche et import de `FormsModule`*

Étape 2 : Résultat

La saisie de « tion » ne conserve que les projets dont le nom contient ce texte (« Application mobile » et « Migration serveur ») :

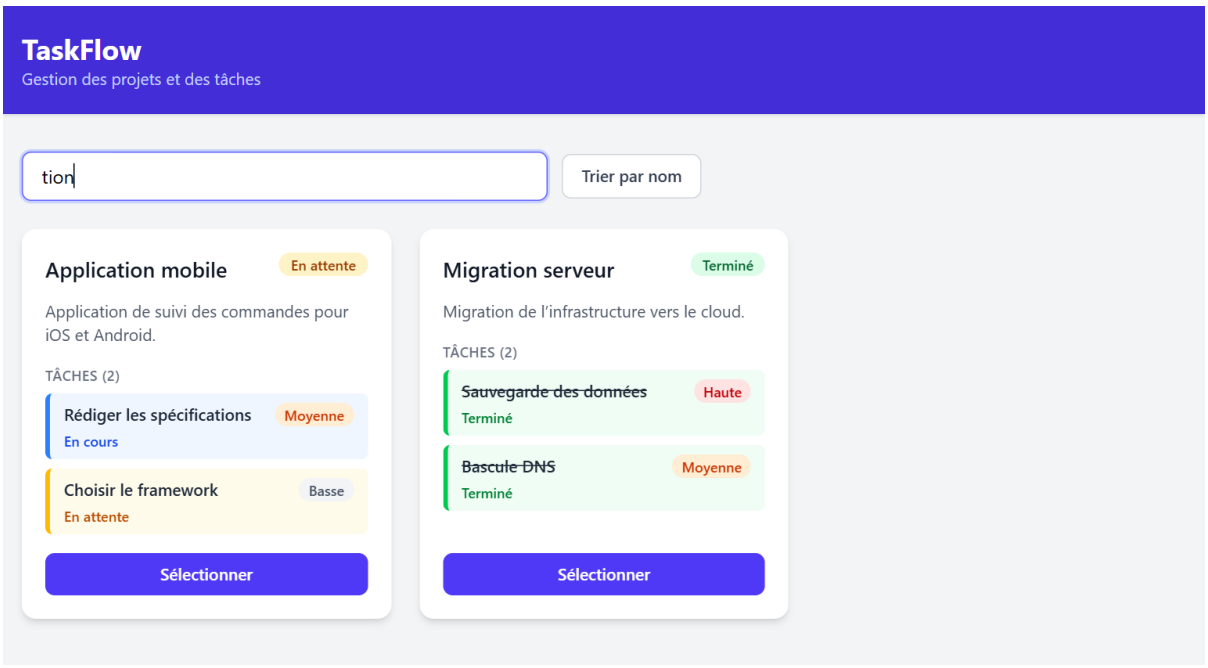


***Figure 22** – Filtrage des projets pendant la saisie*

Si aucun projet ne correspond, le composant `EmptyState` affiche un message :

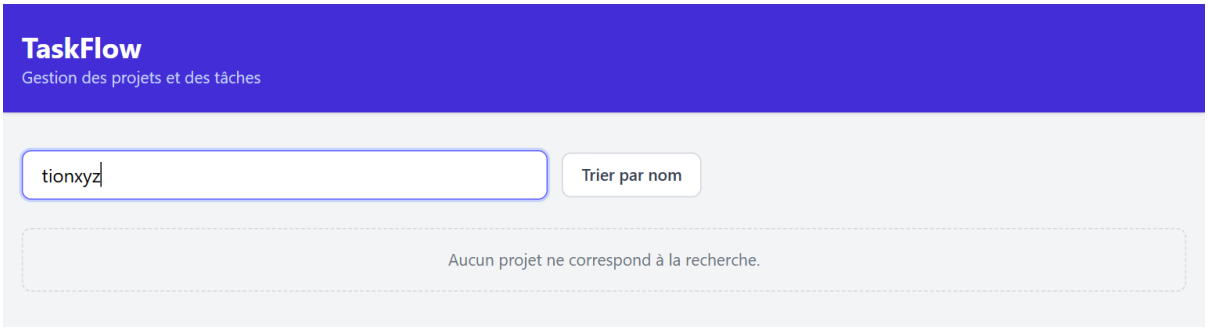


***Figure 23** – Recherche sans résultat*

Partie 8 – Nouvelle syntaxe de contrôle Angular

Étape 1 : Boucle @for

`@for` parcourt `filteredProjects()`. L'expression `track project.id` est obligatoire : elle identifie chaque élément, ce qui permet à Angular de réutiliser les éléments DOM existants au lieu de tout recréer lorsque la liste change (filtrage, tri).

Étape 2 : Conditions @if / @else

`@if` / `@else` affiche soit la grille, soit le message « aucun projet » ; dans `TaskList`, le même principe affiche « Aucune tâche. » lorsque le tableau est vide. La forme `@if (selectedProject(); as selected)` crée en plus un alias local utilisé dans le bandeau de sélection.

project-list.html — src/app/features/projects/components/project-list · lignes 24–36

```
24    @if (filteredProjects().length === 0) {
25      <app-empty-state message="Aucun projet ne correspond à la
    recherche." />
26    } @else {
27      <div class="grid gap-6 sm:grid-cols-2 lg:grid-cols-3">
28        @for (project of filteredProjects(); track project.id) {
29          <app-project-card
30            [project]="project"
31            [selected]="selectedProject()?.id === project.id"
32            (projectSelected)="onProjectSelected($event)"
33          />
34        }
35      </div>
36    }
```

task-list.html — src/app/features/projects/components/task-list · lignes 1–5

```
1  @if (tasks().length === 0) {
2    <app-empty-state message="Aucune tâche." />
3  } @else {
4    <ul class="space-y-2">
5      @for (task of tasks(); track task.id) {
```

Figure 24 – `@for` et `@if` / `@else` dans `project-list.html` et `task-list.html`



Figure 25 – *Projet sans tâche : affichage de « Aucune tâche. »*

Partie 9 – Style avec Tailwind CSS

Étape 1 : Application des consignes

Consigne	Classes Tailwind utilisées
Cartes : fond blanc, ombre, coins arrondis, padding	`bg-white shadow-md rounded-xl p-5`
Tâches avec bordure latérale	`border-l-4` + couleur de bordure selon le statut
Espacement des éléments	`space-y-6`, `space-y-2`, `gap-6`, `mb-3`, `mb-4`, `pt-4`
Statuts différenciés	ambre (En attente), bleu (En cours), vert (Terminé) ; tâches terminées barrées (`line-through`)
Interface responsive	`grid sm:grid-cols-2 lg:grid-cols-3` ; barre de recherche `flex-col sm:flex-row`

project-card.html — src/app/features/projects/components/project-card · lignes 1–5

```
1 <article
2   class="flex h-full flex-col rounded-xl bg-white p-5 shadow-md
  transition"
3   [class.ring-2]="selected()"
4   [class.ring-indigo-500]="selected()"
5 >
```

task-list.html — src/app/features/projects/components/task-list · ligne 6

```
6       <li class="rounded-md border-l-4 px-3 py-2"
  [class]="statusBorder[task.status]">
```

***Figure 26** – Classes de la carte et de la bordure latérale des tâches*

Les classes dépendant du statut ou de la priorité sont centralisées dans des objets typés (`statusBorder`, `statusText`, `priorityBadge`), puis appliquées avec `[class]` ; les classes statiques de l'élément sont conservées. Le type `Record<Status, string>` oblige à définir un style pour **chaque** statut : l'oubli d'un cas est détecté à la compilation.

task-list.ts src/app/features/projects/components/task-list · lignes 13–30

```ts
  protected readonly statusBorder: Record<Status, string> = {
    'En attente': 'border-amber-400 bg-amber-50',
    'En cours': 'border-blue-500 bg-blue-50',
    Terminé: 'border-green-500 bg-green-50',
  };

  protected readonly statusText: Record<Status, string> = {
    'En attente': 'text-amber-700',
    'En cours': 'text-blue-700',
    Terminé: 'text-green-700',
  };

  protected readonly priorityBadge: Record<TaskPriority, string> = {
    Haute: 'bg-red-100 text-red-700',
    Moyenne: 'bg-orange-100 text-orange-700',
    Basse: 'bg-gray-100 text-gray-600',
  };
}
```

***Figure 27** – Correspondance statut / priorité → classes Tailwind dans `TaskList`*

Étape 2 : Rendu responsive

Sur grand écran, les cartes s'affichent sur trois colonnes (figure 8). Sur mobile, la grille passe à une colonne et le bouton de tri se place sous le champ de recherche :

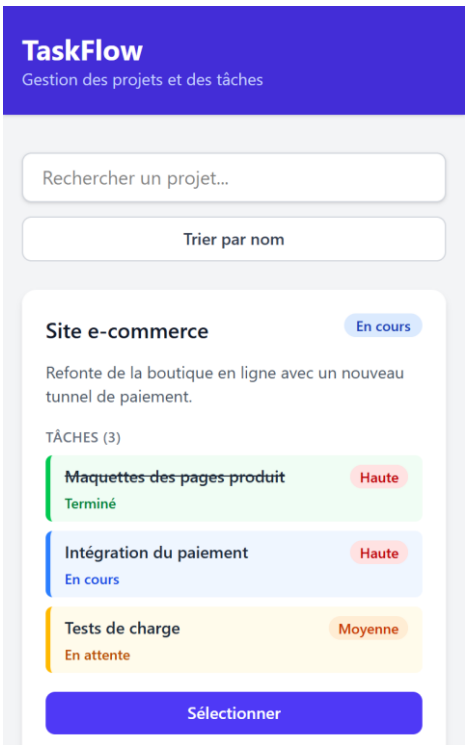


***Figure 28** – Affichage sur mobile (largeur 390 px)*

Partie 10 – Introduction aux Signals

Étape 1 : Remplacer le tableau par un Signal

La liste des projets est stockée dans un signal : `projects = signal<Project[]>(PROJECTS)`. Un signal est une valeur réactive : lorsqu'elle change, Angular met à jour uniquement les parties de la vue qui en dépendent.

Étape 2 : Lecture et modification

- **Lecture** : on appelle le signal comme une fonction, `projects()`.
- **set()** remplace la valeur : `selectedProject.set(project)` lors de la sélection et `projects.set(projects)` à la réception des données (partie 12).
- **update()** calcule la nouvelle valeur à partir de l'ancienne : le bouton « Trier par nom » trie les projets par ordre alphabétique.

project-list.ts — src/app/features/projects/components/project-list · lignes 17–19

```
17   projects = signal<Project[]>(PROJECTS);
18   searchTerm = signal('');
19   selectedProject = signal<Project | null>(null);
```

project-list.ts — src/app/features/projects/components/project-list · lignes 32–39

```
32   onProjectSelected(project: Project): void {
33     console.log(project);
34     this.selectedProject.set(project);
35   }
36
37   sortByName(): void {
38     this.projects.update((projects) => [...projects].sort((a, b) =>
   a.name.localeCompare(b.name)));
39   }
```

***Figure 29** – Déclaration des signals et utilisation de `set()` et `update()`*

Point d'attention : `Array.sort()` modifie le tableau sur place. Nous trions donc une copie (`[...projects]`) afin que `update()` renvoie un **nouveau** tableau : c'est ce changement de référence qui signale la modification à Angular.

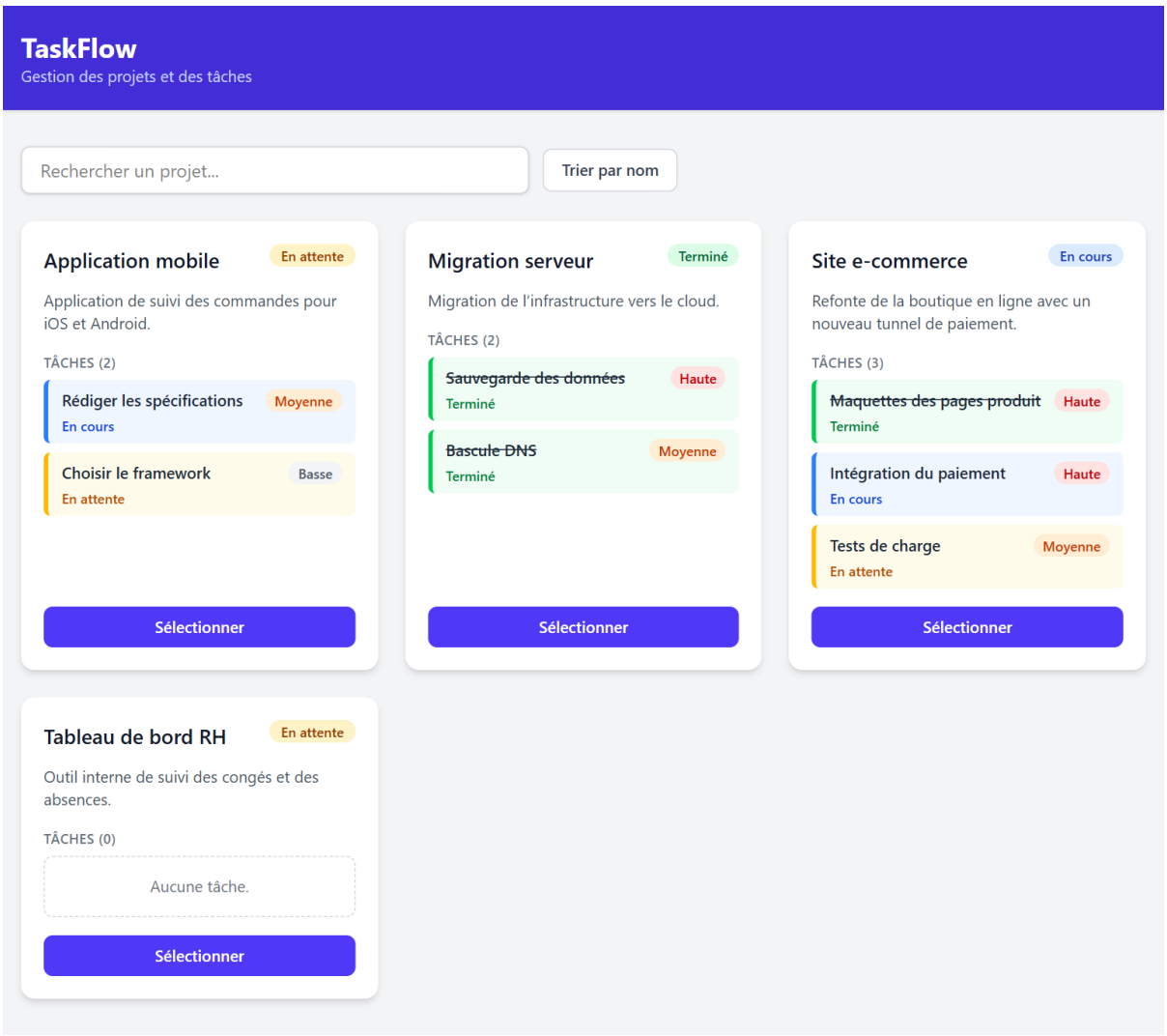


Figure 30 – *Après « Trier par nom » : Application mobile, Migration serveur, Site e-commerce, Tableau de bord RH*

Partie 11 – Computed Signals

Étape 1 : Signal de recherche

Le texte saisi est stocké dans `searchTerm = signal('')`, alimenté par le `[(ngModel)]` de la partie 7.

Étape 2 : Signal calculé filteredProjects

`filteredProjects` est un **computed** : Angular détecte automatiquement qu'il dépend de `searchTerm()` et de `projects()`, et le recalcule uniquement quand l'un des deux change (le résultat est mis en cache entre-temps). La recherche ignore la casse et les espaces en début et fin de saisie. Un computed est en lecture seule : on ne peut pas appeler `set()` dessus.

Étape 3 : Utilisation dans le template

Le template parcourt `filteredProjects()` au lieu de `projects()`. Puisque ce computed dépend aussi de `projects`, le tri et la recherche se combinent sans code supplémentaire.

project-list.ts — src/app/features/projects/components/project-list · lignes 18–24

```
18    searchTerm = signal('');
19    selectedProject = signal<Project | null>(null);
20
21    filteredProjects = computed(() => {
22      const term = this.searchTerm().trim().toLowerCase();
23      return this.projects().filter((project) =>
    project.name.toLowerCase().includes(term));
24    });
```

project-list.html — src/app/features/projects/components/project-list · ligne 28

```
28        @for (project of filteredProjects(); track project.id) {
```

***Figure 31** – Signal `searchTerm`, computed `filteredProjects` et son utilisation dans le template*

Partie 12 – Introduction à RxJS

Étape 1 et 2 : Service et méthode getProjects()

Le service `ProjectService` est créé dans `core/services/`. Grâce à `providedIn: 'root'`, Angular en crée une seule instance pour toute l'application. La méthode `getProjects()` retourne un `Observable<Project[]>`.

Étape 3 : Données statiques avec of()

`of(PROJECTS)` crée un Observable qui émet le tableau une seule fois, de manière synchrone, puis se termine. Le composant ne dépend ainsi que du contrat `Observable<Project[]>` : lorsque les données viendront d'une API avec `HttpClient`, seul le service changera.

project.service.ts — src/app/core/services

```ts
import { Injectable } from '@angular/core';
import { Observable, of } from 'rxjs';
import { PROJECTS } from '../../features/projects/data/projects.data';
import { Project } from '../../features/projects/models/project.model';

@Injectable({ providedIn: 'root' })
export class ProjectService {
  getProjects(): Observable<Project[]> {
    return of(PROJECTS);
  }
}
```

Figure 32 – Service `ProjectService`

Étape 4 : Subscription explicite dans ProjectList

Le service est injecté avec `inject(ProjectService)`. Dans `ngOnInit()`, le composant s'abonne à `getProjects()` et met à jour le signal `projects` avec `set()`. Comme l'Observable créé par `of()` se termine immédiatement, aucune désinscription n'est nécessaire ici.

project-list.ts — src/app/features/projects/components/project-list · lignes 14–15

```ts
export class ProjectList implements OnInit {
  private readonly projectService = inject(ProjectService);
```

project-list.ts — src/app/features/projects/components/project-list · lignes 26–30

```ts
  ngOnInit(): void {
    this.projectService.getProjects().subscribe((projects) => {
      this.projects.set(projects);
    });
  }
```

Figure 33 – Injection du service et subscription dans `ngOnInit()`

Le flux complet de la donnée est donc :

```
PROJECTS → of() → Observable<Project[]> → subscribe() → projects.set() →
filteredProjects (computed) → @for → ProjectCard → TaskList
```

Tests et validation

Compilation

L'application compile sans erreur ni avertissement en mode production :

```
Windows PowerShell

PS C:\...\Angular-TP\TaskFlow> npx ng build
> Building...
✓ Building...
Initial chunk files | Names         |  Raw size | Estimated transfer size
main-LOZOINBB.js    | main          | 258.96 kB |                69.53 kB
styles-WOZTEG23.css | styles        |  14.84 kB |                 3.32 kB

                    | Initial total | 273.80 kB |                72.86 kB

Application bundle generation complete. [1.690 seconds] - 2026-09-
29T09:46:29.418Z

Output location: C:\Users\hasso\OneDrive\Bureau\Angular-
TP\TaskFlow\dist\TaskFlow
```

Figure 34 – Résultat de `ng build`

Tests unitaires

Le test généré par Angular CLI vérifiait le texte « Hello, TaskFlow » de la page d'accueil par défaut ; il a été adapté au nouveau titre. Les deux tests passent :

```
Windows PowerShell

PS C:\...\Angular-TP\TaskFlow> npx ng test --watch=false
> Building...
✓ Building...
Application bundle generation complete. [1.257 seconds] - 2026-09-
29T09:46:32.036Z


 RUN  v5.0.2 C:/Users/hasso/OneDrive/Bureau/Angular-TP/TaskFlow


 Test Files  1 passed (1)
      Tests  2 passed (2)
   Start at  10:46:32
   Duration  1.20s (environment 68%, setup 18%, tests 5%, transform 5%, import
3%, worker 1%)
```

Figure 35 – Résultat de `ng test`

Vérification fonctionnelle

Fonctionnalité testée	Résultat attendu	Résultat
Affichage initial	4 cartes avec statut, description et tâches	Conforme (fig. 8)
Recherche « tion »	2 projets affichés	Conforme (fig. 22)
Recherche « xyz »	Message « Aucun projet… »	Conforme (fig. 23)
Bouton « Sélectionner »	Bandeau, carte encadrée, projet affiché en console	Conforme (fig. 20)
Projet sans tâche	Message « Aucune tâche. »	Conforme (fig. 25)
Bouton « Trier par nom »	Ordre alphabétique	Conforme (fig. 30)
Affichage mobile	Une seule colonne	Conforme (fig. 28)

Difficultés rencontrées

Lors des premières compilations, le processus Node.js s'est arrêté avec l'erreur « process out of memory » : la machine ne disposait plus d'assez de mémoire virtuelle. Il ne s'agissait pas d'une erreur dans le code (la vérification TypeScript passait). Après libération de mémoire, la compilation et les tests se sont déroulés normalement.

Conclusion

Ce TP nous a permis de construire une première application Angular complète et structurée. Nous avons organisé le code selon une architecture feature-based, découpé l'interface en composants Standalone réutilisables et fait communiquer ces composants avec `input()` et `output()`.

Nous avons utilisé les principaux types de Data Binding, la nouvelle syntaxe `@if` / `@for`, et Tailwind CSS pour obtenir une interface claire et responsive. Enfin, les Signals (`signal()`, `computed()`) nous ont permis de gérer l'état local de façon réactive, et une première utilisation de RxJS (`of()`, `subscribe()`) a montré comment un service alimente cet état.

Pour la suite, l'application pourra évoluer vers l'architecture scalable présentée dans l'énoncé : ajout du routage et de pages de détail, récupération des données depuis une API avec `HttpClient`, et centralisation de l'état dans un store (`project.store.ts`).